Algorithmic Problem Solving 2026

Project Report

Group	Group H
Members	Karl Thedor Ruby (krub@itu.dk) Kristoffer Mejborn Eliasson (krme@itu.dk) Lukas Shaghashvili-Johannessen (lush@itu.dk)
Supervisor	Holger Dell, Martin Aumüller
Course	Algorithmic Problem Solving (APS 2026)
Date	[YYYY-MM-DD]

IT University of Copenhagen

1. Abstract

Provide a short summary (max 200 words) describing the designed problem, the solved problems, and the main algorithmic techniques used.

2. Designed Problem

Provide a self-contained problem that can be used for automated judging. Make sure `verifyproblem` accepts your files.

2.1. Problem description

Give the problem statement as you would present it to contestants. Include input and output formats, constraints, and examples.

2.2. Intended solution

Describe the algorithmic idea behind an intended (accepted) solution. Use pseudocode and a short complexity analysis (time and memory). Example:

```
function solve(input):
  parse input
  // algorithmic steps
  return output
```

- **Time complexity:** O(…)
- **Memory:** O(…)

2.3. Test-case design

Explain how you generated sample and secret inputs. For each test group, state the purpose, parameter ranges, and expected corner cases.

- **Sample tests:** small cases for manual checking.
- **Secret tests:** randomized and worst-case instances to exercise complexity.

2.4. Accepted and wrong solutions

Include at least one accepted solution and one or more wrong/inefficient variants. For each wrong solution, explain why it fails (e.g., incorrect logic, TLE, memory).

2.5. Input validation & formatting

Specify how your judge should treat malformed input and which assumptions contestants may rely on.

2.6. Verify notes

Confirm that `verifyproblem` runs cleanly. If it produced warnings, document them and the fixes applied.

2.7. Testing and Validation

- Describe your testing strategy
- Provide commands to reproduce tests.

Example commands:

2.8. Source for Designed Problem

Provide a compact description of the repository layout and how to run the judge and tests locally.

- **Files to include in zip:** problem statement, solutions, test generator(s), verifyproblem configuration, README.
- **Run locally:** provide exact commands used to generate and run tests.

3. Solved Problems

3.1. Robert Hood

For each problem you solved (one per group member as a rule of thumb), include the following subsections.

3.1.1. Problem metadata

- **Problem name / Kattis link:** [URL]
- **Short formal I/O description:** (not a repeat of the original prose)

3.1.2. Solution overview

Explain the chosen algorithm, key data structures, and why this approach was selected.

3.1.3. Correctness argument

Sketch a correctness proof or invariants that guarantee the solution works for all valid inputs.

3.1.4. Complexity and time-limit reasoning

Provide asymptotic runtime and memory usage. Discuss how your implementation meets the problem limits and expected worst-case inputs.

3.1.5. Empirical worst-case testing

If possible, generate or describe inputs that approximate worst-case behaviour, measure running time, and report results.

3.1.6. Implementation notes

- **Language used:** e.g. C++17, Python 3.10
- **Files:** path/to/solution
- **How to run:** `./run_solution input.txt`

3.2. Cookie Selection

For each problem you solved (one per group member as a rule of thumb), include the following subsections.

3.2.1. Problem metadata

- **Problem name / Kattis link:** [URL]
- **Short formal I/O description:** (not a repeat of the original prose)

3.2.2. Solution overview

Explain the chosen algorithm, key data structures, and why this approach was selected.

3.2.3. Correctness argument

Sketch a correctness proof or invariants that guarantee the solution works for all valid inputs.

3.2.4. Complexity and time-limit reasoning

Provide asymptotic runtime and memory usage. Discuss how your implementation meets the problem limits and expected worst-case inputs.

3.2.5. Empirical worst-case testing

If possible, generate or describe inputs that approximate worst-case behaviour, measure running time, and report results.

3.2.6. Implementation notes

- **Language used:** e.g. C++17, Python 3.10
- **Files:** path/to/solution
- **How to run:** `./run_solution input.txt`

3.3. Robert Hood

For each problem you solved (one per group member as a rule of thumb), include the following subsections.

3.3.1. Problem metadata

- **Problem name / Kattis link:** [URL]
- **Short formal I/O description:** (not a repeat of the original prose)

3.3.2. Solution overview

Explain the chosen algorithm, key data structures, and why this approach was selected.

3.3.3. Correctness argument

Sketch a correctness proof or invariants that guarantee the solution works for all valid inputs.

3.3.4. Complexity and time-limit reasoning

Provide asymptotic runtime and memory usage. Discuss how your implementation meets the problem limits and expected worst-case inputs.

3.3.5. Empirical worst-case testing

If possible, generate or describe inputs that approximate worst-case behaviour, measure running time, and report results.

3.3.6. Implementation notes

- **Language used:** e.g. C++17, Python 3.10
- **Files:** path/to/solution
- **How to run:** `./run_solution input.txt`

4. Appendix A — Checklist

- [ ] Problem description included
- [ ] Intended solution described
- [ ] Test-case design documented
- [ ] verifyproblem passes
- [ ] Source zip prepared
- [ ] Solved problems documented (one per member)

5. Appendix B — Code listings

Include important code snippets or point to files under `solutions/` and `tests/`.